



Signals

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Signals and Asynchronous Events

Signals are a mechanism used by the operating system to notify a process that an event has occurred. Signals are **asynchronous**. Unlike a normal function call, the process does not control when a signal arrives. The operating system delivers a signal whenever a relevant event occurs.

Typical situations that generate signals include:

- terminal input such as `Ctrl+C`,
- expiration of a timer,
- termination of a child process,
- invalid memory access,
- another process explicitly sending a signal.

When a signal is delivered, the process may terminate, ignore the signal, or execute a user defined handler.

Signals notify a process that an event occurred.

They are not synchronization primitives and are not intended to protect shared data like mutexes or semaphores.

1.1 Basic Signal Concepts

Each signal has three main properties:

- a **name** such as `SIGINT`, `SIGTERM`, or `SIGALRM`,
- a **numeric identifier** defined in `<signal.h>`,
- a **default action** performed by the operating system.

For most signals, a process can choose one of three behaviors:

- **Default action:** The operating system performs the standard behavior associated with the signal.
- **Ignoring the signal:** The signal is discarded and no action is taken.
- **Catching the signal:** The process installs a signal handler function that executes when the signal arrives.

Some signals cannot be caught or ignored:

- `SIGKILL`
- `SIGSTOP`

These signals are always handled directly by the kernel.

1.2 Types of Signals

Signals can be divided into two groups: **system signals** and **user-defined signals**, depending on their origin.

System Signals

System signals are predefined by the operating system and are generated by the kernel or hardware exceptions.

Examples include:

- **SIGINT** interrupt signal generated by **Ctrl+C**
- **SIGTERM** request for graceful termination
- **SIGKILL** immediate process termination
- **SIGSEGV** invalid memory access
- **SIGALRM** timer expiration
- **SIGCHLD** child process changed state
- **SIGSTOP** stop execution of a process for later resumption
- **SIGCONT** resume execution of a stopped process

These signals are generated automatically by the operating system.

User Defined Signals

Applications can also send signals to communicate between processes.

POSIX defines two signals reserved for application specific use:

- **SIGUSR1**
- **SIGUSR2**

These signals have no predefined meaning. Programs may assign their own semantics.

Typical uses include:

- requesting a process to reload configuration files
- triggering a status report
- coordinating behavior between cooperating processes

1.3 Handling Signals

1.3.1 Basic Signal Handler

The following example demonstrates how a program can react to a signal using a **signal handler**. A signal handler is a function that is executed when a specific signal is delivered to the process.

C Code Example: 01_basic_signal_handler.c

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void handler(int signo) {
6     printf("Signal received\n");
7 }
8
9 int main() {
10     signal(SIGINT, handler);
11     while (1)
12         pause();
13 }
```

Testing the Program

Execute the program and press **Ctrl+C** in the terminal. The key combination **Ctrl+C** generates the signal **SIGINT**.

Program Output

```
^CSignal received
```

Explanation

- The function `handler()` is the signal handler.
- The call `signal(SIGINT, handler)` installs the handler for the signal **SIGINT**.
- When the user presses **Ctrl+C**, the signal **SIGINT** is sent to the process.
- The handler function is executed and prints the message.
- The function `pause()` suspends the program until a signal arrives.

Because **SIGINT** is handled, **Ctrl+C** no longer terminates the program.

To stop the program, use **Ctrl+** (sends **SIGQUIT**) or run `kill -9 <pid>` from another terminal.

1.3.2 Handling Multiple Signals

The next example shows how a program can react differently to multiple signals. In this case the program handles the user-defined signals **SIGUSR1** and **SIGUSR2**.

C Code Example: 02_multiple_signals.c

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void handler(int signo) {
6
7     if (signo == SIGUSR1)
8         printf("SIGUSR1 received\n");
9
10    if (signo == SIGUSR2)
11        printf("SIGUSR2 received\n");
12 }
13
14 int main() {
15
16     signal(SIGUSR1, handler);
17     signal(SIGUSR2, handler);
18
19     printf("PID: %d\n", getpid());
20
21     while (1)
22         pause();
23 }
```

Sending Signals

Open another terminal and send signals to the running program:

```
kill -USR1 <pid>
kill -USR2 <pid>
```

Replace `<pid>` with the process ID printed by the program.

Terminal Output

```
SIGUSR1 received
SIGUSR2 received
```

Explanation

- The program installs a handler for two signals: `SIGUSR1` and `SIGUSR2`.
- The function `getpid()` prints the process ID so another process can send signals to it.
- The command `kill` sends a signal to a process.
- When the signal arrives, the handler checks which signal was received and prints the corresponding message.
- The function `pause()` keeps the program waiting for signals.

The signals `SIGUSR1` and `SIGUSR2` are reserved for application-defined behavior and are commonly used for interprocess communication.

1.4 Creating a Signal Set

In the previous examples we installed signal handlers and reacted to signals such as `SIGINT`, `SIGUSR1`, and `SIGUSR2`. Before introducing signal blocking, we first need a way to represent a **set of signals**.

The type `sigset_t` represents a collection of signals. Functions such as `sigemptyset()` and `sigaddset()` are used to build this set.

C Code Example: `03_signal_set.c`

```
1 #include <signal.h>
2 #include <stdio.h>
3
4 int main() {
5
6     sigset_t set;
7
8     sigemptyset(&set);
9     sigaddset(&set, SIGINT);
10
11     printf("Signal set created\n");
12
13     return 0;
14 }
```

Explanation

- `sigset_t` represents a set of signals.
- `sigemptyset()` initializes an empty signal set.
- `sigaddset(&set, SIGINT)` adds the signal `SIGINT` to the signal set `set`, meaning this signal becomes part of that set for later operations such as blocking or unblocking.
- In this example the set contains only the signal `SIGINT`.

Signal sets are used when a program needs to manipulate multiple signals, for example when blocking or unblocking signals.

1.5 Blocking and Unblocking a Signal

Now that we know how to create a signal set, we can use it to temporarily **block a signal**. Blocking prevents the signal from being delivered immediately.

If a blocked signal occurs, it becomes **pending** and will be delivered later when the signal is unblocked.

C Code Example: 04_block_unblock_signal.c

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void handler(int signo) {
6     printf("SIGINT received\n");
7 }
8
9 int main() {
10
11     sigset_t set;
12
13     signal(SIGINT, handler);
14
15     sigemptyset(&set);
16     sigaddset(&set, SIGINT);
17
18     sigprocmask(SIG_BLOCK, &set, NULL);
19
20     printf("SIGINT blocked for 5 seconds\n");
21     sleep(5);
22
23     sigprocmask(SIG_UNBLOCK, &set, NULL);
24
25     printf("SIGINT unblocked\n");
26
27     sleep(5);
28 }
```

`sigprocmask` function controls which signals are blocked for the current process (or thread).

`sigprocmask(how, set, oldset)` takes three arguments: the operation to perform (such as `SIG_BLOCK`), the set of signals to apply the operation to, and an optional pointer to store the previous signal mask.

How to Test

- Run the program.
- During the first 5 seconds, press `Ctrl+C`.
- The signal `SIGINT` is generated but cannot be delivered because it is blocked.
- After 5 seconds the signal becomes unblocked and the pending signal is delivered.

Possible Program Outputs

Depending on when `Ctrl+C` is pressed, the behavior differs:

- Pressing **Ctrl+C** during the blocking period:

```
SIGINT blocked for 5 seconds
^CSIGINT received
SIGINT unblocked
```

If the signal occurs while it is blocked, it is not lost. The signal is blocked, becomes pending, and is delivered immediately after it is unblocked.

- Not pressing **Ctrl+C**:

```
SIGINT blocked for 5 seconds
SIGINT unblocked
```

No signal is generated, so the program runs normally.

- Pressing **Ctrl+C** after unblocking (during second sleep):

```
SIGINT blocked for 5 seconds
SIGINT unblocked
^CSIGINT received
```

If **Ctrl+C** is pressed during the second `sleep()`, the signal is delivered immediately and interrupts `sleep()`. The handler is executed, and then the program continues normally and reaches the end of `main()`. Thus, the program terminates due to normal execution, not because of the signal.

1.6 Why `signal()` Is Not Ideal

In earlier examples we used the function `signal()` to install a signal handler. Historically, this interface behaves differently across UNIX systems. For example, some systems automatically reset the handler after a signal is delivered, while others keep the handler installed.

Because of these differences, modern programs prefer the function `sigaction()`, which provides a more reliable and portable interface for signal handling.

Advantages of `sigaction()` include:

- consistent signal handling semantics across UNIX systems
- explicit control over which signals are blocked during execution of a handler
- optional automatic restart of interrupted system calls
- greater flexibility through additional configuration flags

Installing a Signal Handler Using `sigaction`

The following example installs a handler for the signal `SIGINT` using `sigaction()`. The program waits for signals and reacts when the user presses **Ctrl+C**.

C Code Example: 05_sigaction_handler.c

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void handler(int signo) {
6     printf("caught SIGINT\n");
7 }
8
9 int main() {
10
11     struct sigaction act;
12
13     act.sa_handler = handler;
14     sigemptyset(&act.sa_mask);
15     act.sa_flags = 0;
16
17     sigaction(SIGINT, &act, NULL);
18
19     while (1)
20         pause();
21 }
```

Testing the Program

Press **Ctrl+C** a few times while the program is running. The key combination **Ctrl+C** generates the signal **SIGINT**.

Each time **Ctrl+C** is pressed, the handler function is executed and prints the message.

The structure `struct sigaction` describes how a signal should be handled.

The field `sa_handler` specifies the function that will run when the signal arrives.

The function `sigemptyset()` initializes the signal mask `act.sa_mask` to an empty set, which is used during handler execution. Signals in this mask are temporarily blocked while the handler runs (the signal being handled is blocked automatically during handler execution).

The field `sa_flags` allows additional behavior to be configured. In this example it is set to 0, meaning no special options are enabled. Some commonly used flags include:

- **SA_RESTART** Automatically restarts certain interrupted system calls (such as `read()`, `wait()`, or `pause()`) after the signal handler finishes.
- **SA_SIGINFO** Provides extended information about the signal. When this flag is used, the handler receives additional parameters containing details such as the sending process ID and other signal data.

The call `sigaction(SIGINT, &act, NULL)` installs the handler configuration for the signal `SIGINT`.

The function `pause()` suspends the program until a signal is delivered.

Program Output

```
^Ccaught SIGINT
^Ccaught SIGINT
^Ccaught SIGINT
^Ccaught SIGINT
```

1.7 Interrupted System Calls

Signals may interrupt blocking system calls such as `read()`, `wait()`, `sleep()`, or `pause()`.

If a signal arrives while such a call is waiting, the system call may stop and return an error. In this case the global variable `errno` is set to

```
1 errno == EINTR
```

This indicates that the system call was interrupted by a signal.

Example Pattern

A common programming pattern is to restart the system call when `EINTR` occurs.

```
1 ssize_t n;
2
3 again:
4 n = read(fd, buf, sizeof(buf));
5
6 if (n < 0) {
7     if (errno == EINTR)
8         goto again;
9 }
```

If the call is interrupted, the program jumps back and tries the operation again.

C Code Example: `06_eintr_read.c`

The following example shows how a blocking system call can be interrupted by a signal.

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <errno.h>
5
6 void handler(int signo) {
7     printf("Signal received\n");
8 }
9
10 int main() {
```

```
11     struct sigaction act;
12
13     act.sa_handler = handler;
14     sigemptyset(&act.sa_mask);
15     act.sa_flags = 0;    // important: no SA_RESTART
16
17     sigaction(SIGINT, &act, NULL);
18
19     printf("Waiting for input...\n");
20
21     char buf[100];
22     ssize_t n = read(STDIN_FILENO, buf, sizeof(buf));
23
24     if (n < 0 && errno == EINTR) {
25         printf("read interrupted by signal\n");
26     }
27
28     printf("Program continues\n");
29 }
```

Testing

While the program waits for input, press **Ctrl+C**.

Example Output

```
Waiting for input...
^CSignal received
read interrupted by signal
Program continues
```

When a signal arrives during `read()`, two outcomes are possible:

- The call is automatically restarted (for example when `SA_RESTART` is used), so `read()` continues waiting.
- The call is interrupted and returns early with `errno == EINTR`.

If `read()` is interrupted, the program can detect `EINTR` and decide whether to retry the operation.

1.8 SIGALRM and the `alarm()` Function

`SIGALRM` is a signal used for timers, and the OS sends it to a process when a timer expires, which terminates the process if it is not handled. The function `alarm(seconds)` schedules the signal `SIGALRM` to be delivered to the process after a specified number of seconds. If the process is still running when the timer expires, the operating system sends the signal `SIGALRM`. The program can install a signal handler to react to this event.

C Code Example: 07_alarm_timeout.c

The following program sets a timer for 5 seconds. If the timer expires, the signal handler prints a message and terminates the program.

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 void timeout_handler(int signo) {
7     printf("timeout occurred\n");
8     exit(1);
9 }
10
11 int main() {
12     signal(SIGALRM, timeout_handler);
13     alarm(5);
14     printf("You have 5 seconds...\n");
15     pause();
16     return 0;
17 }
```

Program Output

```
You have 5 seconds...
timeout occurred
```

After 5 seconds, the signal `SIGALRM` is delivered and the handler function is executed.

Explanation

- The function `signal(SIGALRM, timeout_handler)` installs a handler for the signal `SIGALRM`.
- The call `alarm(5)` schedules the signal to be delivered after 5 seconds.
- The program prints a message and then waits using `pause()`.
- When the timer expires, the signal `SIGALRM` is sent to the process.
- The handler function `timeout_handler()` runs and terminates the program using `exit()`.

`alarm()` is commonly used to implement timeouts for operations that may block indefinitely, such as waiting for input or network communication.

1.9 Signal Handlers and Reentrancy

Signal handlers execute asynchronously with respect to the main program. This means a handler may run at almost any point during program execution. Because of this, many library functions are not safe to call inside signal handlers. If a signal interrupts one of these functions and the

same function is called again inside the signal handler, the program may behave incorrectly because the function's internal state may become inconsistent.

Unsafe examples include:

- `printf()`
- `malloc()`
- many other standard library functions

If such a function is interrupted by a signal and then called again inside the handler, the program may behave incorrectly.

A safer approach is:

- set a global flag of type `sig_atomic_t`
- return immediately from the handler
- perform the actual work in the main program

The type `sig_atomic_t` is guaranteed to be read and written atomically, which makes it safe to use in signal handlers.

C Code Example: `08_signal_flag.c`

```
1 #include <signal.h>
2 #include <stdio.h>
3
4 volatile sig_atomic_t got_sigint = 0;
5
6 void handler(int signo) {
7     got_sigint = 1;
8 }
9
10 int main() {
11
12     signal(SIGINT, handler);
13
14     while (1) {
15         if (got_sigint) {
16             printf("SIGINT observed in main loop\n");
17             got_sigint = 0;
18         }
19     }
20 }
```

Testing

Press `Ctrl+C` while the program is running.

Program Output

```
^CSIGINT observed in main loop
^CSIGINT observed in main loop
```

Each time the signal `SIGINT` is generated, the handler sets the flag `got_sigint`. The main loop checks this flag and performs the actual work.

Explanation

- The variable `got_sigint` is declared as `volatile sig_atomic_t`, which makes it safe to modify inside a signal handler.
- When `SIGINT` occurs, the handler sets the flag to 1.
- The handler performs minimal work and returns immediately.
- The main loop checks the flag and prints a message when the signal has been observed.

Signal handlers should do as little work as possible. A common rule is to set a flag and let the main program handle the rest.

1.10 Signals and Child Processes

When a child process terminates, the operating system sends the signal `SIGCHLD` to the parent process. The parent process can handle this signal and use `wait()` or `waitpid()` to collect the child's termination status.

C Code Example: `09_sigchld_reap.c`

The following program creates a child process using `fork()`. When the child terminates, the parent receives the signal `SIGCHLD` and the signal handler calls `wait()` to reap the child.

```
1 #include <signal.h>
2 #include <sys/wait.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 void child_handler(int signo) {
8     int status;
9     wait(&status);
10    printf("child terminated\n");
11 }
12
13 int main() {
14     signal(SIGCHLD, child_handler);
15     pid_t pid = fork();
16     if (pid == 0) {
17         sleep(2);
18         exit(0);
```

```
19     }  
20     while (1)  
21         pause();  
22 }
```

Program Output

child terminated

If a parent process does not call `wait()` or `waitpid()` after a child terminates, the child becomes a **zombie process**. Zombie processes remain in the system until their parent collects their exit status. So handling `SIGCHLD` signal and using `wait()` (or `wait_pid()`) inside the handler removes the zombie process.